

Министерство науки и высшего образования Российской Федерации
Муромский институт (филиал)
федерального государственного бюджетного образовательного учреждения
высшего образования
**«Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»**
(МИ ВлГУ)

Факультет информационных технологий радиоэлектроники
Кафедра информационных систем

КУРСОВАЯ РАБОТА

по курсу Управление данными
на тему: Разработка программы для визуализации работы алгоритмов
сортировки

Руководитель

(уч. степень, звание)

Метелкин А. С.

(фамилия, инициалы)

(подпись)

(дата)

Члены комиссии

Студент ИС-123

(группа)

Смакотина Д. В.

(фамилия, инициалы)

(подпись)

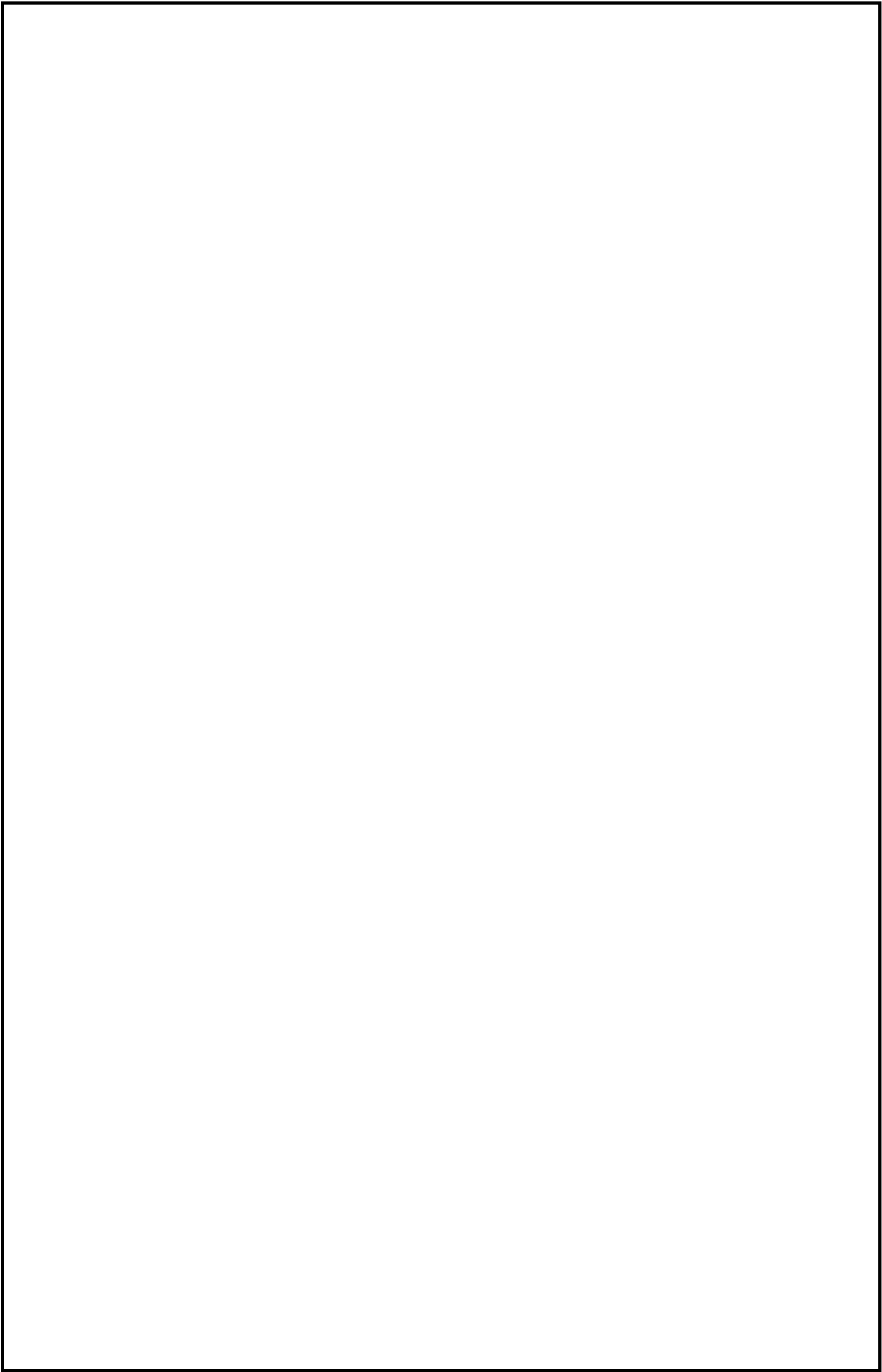
(Ф.И.О.)

(подпись)

(Ф.И.О.)

(подпись)

(дата)

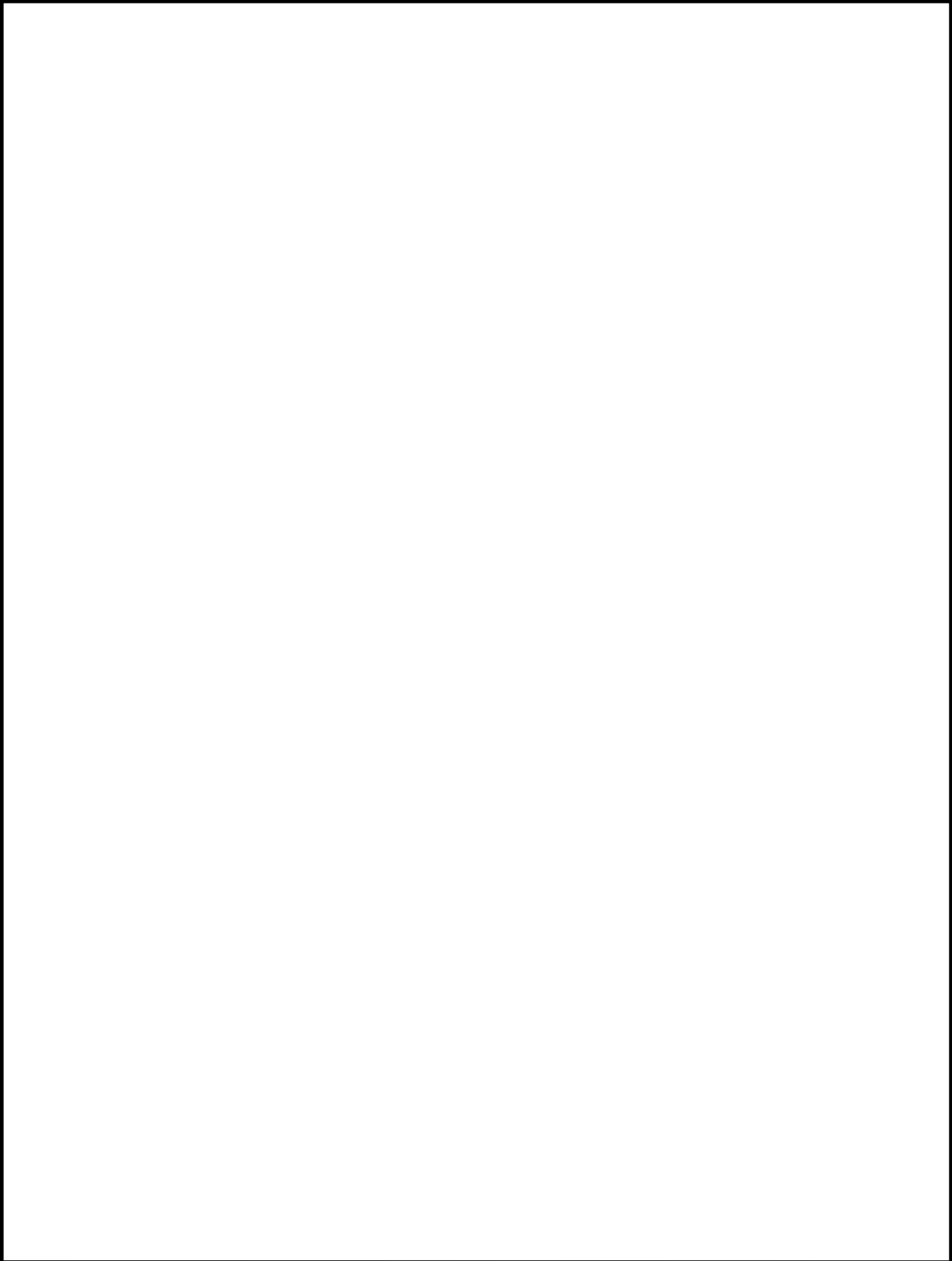


В курсовой работе отражена разработка приложения для визуализации алгоритмов сортировки, реализована обработка данных с использованием файлов различных форматов (TXT, CSV, JSON). Спроектирована архитектура программы на основе принципов объектно-ориентированного программирования с применением паттернов проектирования. Создан интерфейс, позволяющий загружать данные, выбирать алгоритм сортировки, наблюдать процесс выполнения и сохранять результат в файл. Реализованы алгоритмы сортировки: пузырьковая, быстрая и сортировка выбором, а также различные способы визуализации: консольная, JavaFX и Swing. Проведено тестирование основных модулей. Разработанное приложение обеспечивает наглядное представление работы алгоритмов и удобное взаимодействие пользователя.

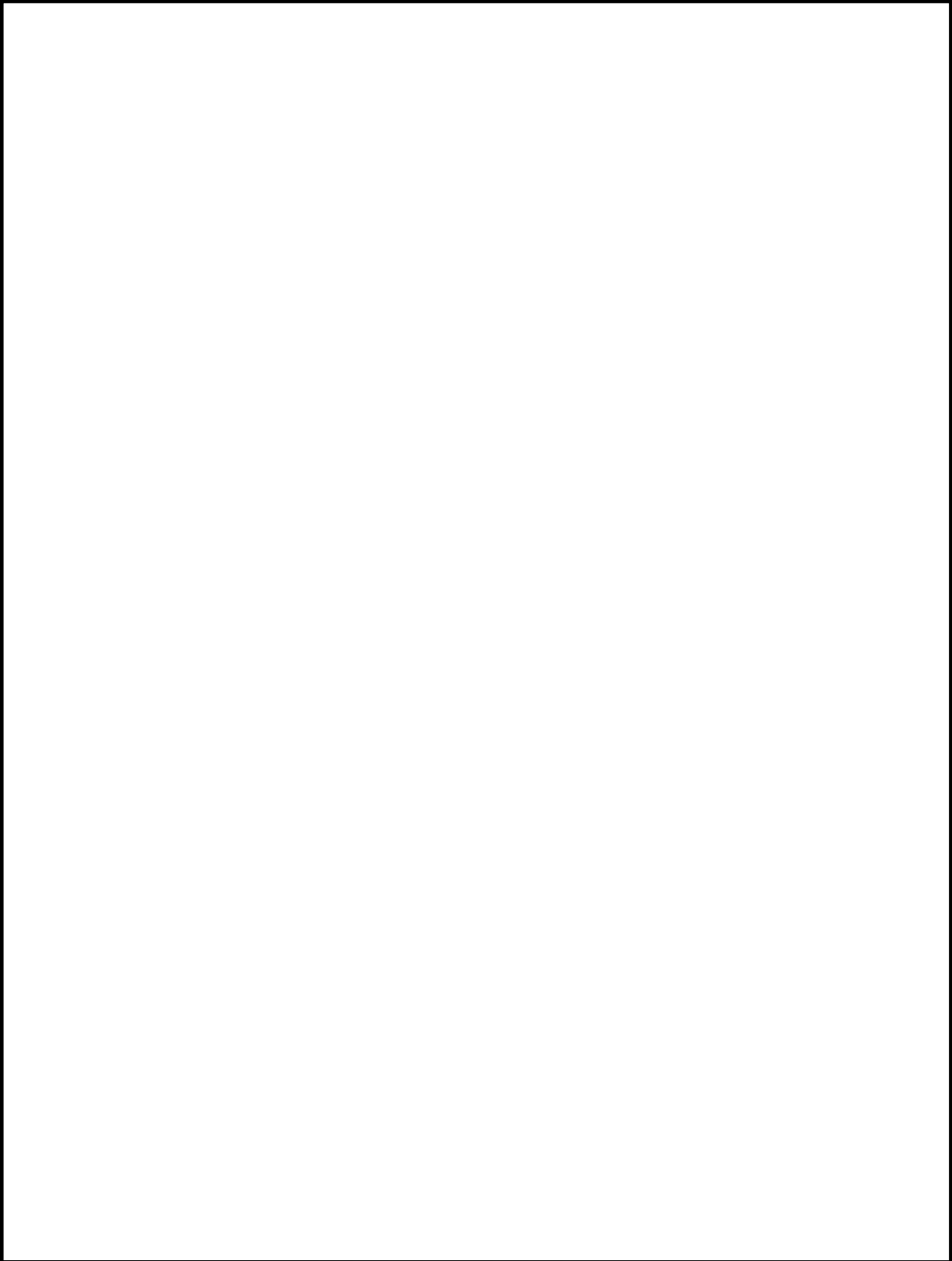
Содержание

Введение	5
1. Анализ технического задания	7
2. Проектирование программы	10
2.1 Проектирование системы на Java	11
2.2 Проектирование системы хранения данных	12
2.3 Проектирование графического интерфейса	13
3. Разработка программы.....	16
3.1 Разработка системы на Java	16
3.2 Разработка системы хранения данных	22
3.3 Разработка графического интерфейса.....	24
4. Тестирование	29
Заключение	34
Библиография	36

					МИВУ.09.03.02 020 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата				
Разраб.		Смакотина Д. В.			Разработка программы для визуализации работы алгоритмов сортировки	Лит.	Лист	Листов
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								



					МИВУ.09.03.02 020 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка программы для визуализации работы алгоритмов сортировки	Лит.	Лист	Листов
Разраб.		Смакотина Д. В.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								



					МИВУ.09.03.02 020 ПЗ			
Изм.	Лист	№ докум.	Подпись	Дата	Разработка программы для визуализации работы алгоритмов сортировки	Лит.	Лист	Листов
Разраб.		Смакотина Д. В.						
Провер.		Метелкин А.С.					4	
Реценз.						ИС-123		
Н. Контр.								
Утверд.								

Введение

В современных условиях изучение алгоритмов и структур данных является важной частью подготовки специалистов в области программирования. Одной из ключевых задач является понимание принципов работы алгоритмов сортировки, которые широко применяются при обработке данных. Для более наглядного изучения и анализа таких алгоритмов используются программные средства визуализации, позволяющие наблюдать процесс их выполнения в реальном времени. Подобные решения способствуют лучшему усвоению материала и повышают эффективность обучения.

Целью данного курсового проекта является разработка программного приложения для визуализации алгоритмов сортировки. Программа должна обеспечивать загрузку данных из файлов, выбор алгоритма сортировки, отображение процесса выполнения с использованием различных способов визуализации, а также сохранение результата в файл. Особое внимание уделяется построению архитектуры приложения с использованием интерфейсов и паттернов проектирования, обеспечивающих гибкость и расширяемость системы.

Для достижения поставленной цели в рамках проекта необходимо решить следующие задачи:

- разработать архитектуру приложения с использованием интерфейсов для разделения логики программы;
- реализовать алгоритмы сортировки (пузырьковая, быстрая, сортировка выбором);
- обеспечить возможность выбора алгоритма, способа хранения данных и варианта визуализации;
- реализовать загрузку и сохранение данных в различных форматах (ТХТ, CSV, JSON)
- разработать несколько способов визуализации;

					МИВУ.09.03.02 020 ПЗ	Лист
						5
Изм.	Лист	№ докум.	Подпись	Дата		

- обеспечить возможность расширения программы для добавления новых алгоритмов и модулей.

Результатом работы является программное приложение, реализующее визуализацию алгоритмов сортировки с возможностью выбора алгоритма, формата данных и способа отображения. Разработанная система демонстрирует применение паттернов проектирования и принципов объектно-ориентированного программирования, обеспечивает наглядное представление процесса сортировки и может использоваться в учебных целях.

					МИВУ.09.03.02 020 ПЗ	Лист
						6
Изм.	Лист	№ докум.	Подпись	Дата		

1. Анализ технического задания

Разработка приложения для визуализации алгоритмов сортировки требует учёта особенностей предметной области, связанной с обработкой и отображением данных, а также с демонстрацией работы алгоритмов в реальном времени. В образовательной и практической деятельности подобные программы используются для наглядного представления процессов сортировки, что позволяет лучше понять принципы их работы и сравнить эффективность различных алгоритмов. В связи с этим при проектировании приложения особое внимание уделяется разделению логики сортировки и визуализации, а также обеспечению гибкости и расширяемости системы.

В качестве существующих аналогов можно выделить такие проекты, как Visualgo, Sorting Algorithm Visualizer и Algorithm Visualizer. Данные системы предназначены для демонстрации работы алгоритмов сортировки и визуализации структур данных. Большинство подобных решений реализуются как web-приложения с использованием технологий JavaScript, HTML5 Canvas, React или D3.js для построения анимации и отображения графических элементов. Основным преимуществом web-ориентированных решений является удобство запуска через браузер и высокая интерактивность интерфейса. Однако подобные приложения зачастую ориентированы исключительно на визуализацию и не предусматривают гибкой работы с различными форматами хранения данных или возможности запуска программы в нескольких режимах визуализации.

В отличие от существующих аналогов, в рамках данной работы было принято решение реализовать настольное приложение на языке Java с использованием нескольких технологий визуализации. В качестве основной платформы разработки использовался язык программирования Java 17. Для реализации графических интерфейсов были применены библиотеки Swing и JavaFX. Swing использовался для создания классического графического интерфейса визуализации, а JavaFX — для разработки более современной

					МИВУ.09.03.02 020 ПЗ	Лист
						7
Изм.	Лист	№ докум.	Подпись	Дата		

интерактивной визуализации с анимацией и элементами управления скоростью сортировки. Дополнительно была реализована консольная визуализация, обеспечивающая отображение процесса сортировки в текстовом режиме.

Для сборки проекта и управления зависимостями использовалась система Maven. В качестве инструментов тестирования и анализа качества программного кода применялись JUnit 5, JaCoCo и Checkstyle. JUnit 5 использовался для создания модульных тестов, JaCoCo — для анализа покрытия кода тестами, а Checkstyle — для статического анализа исходного кода и проверки соответствия стилю программирования. Для автоматизации сборки и тестирования проекта была настроена система GitHub Actions.

Предметная область включает обработку числовых данных, представленных в виде массивов, а также их визуализацию различными способами. Исходные данные поступают из файлов различных форматов (ТХТ, CSV, JSON) и преобразуются во внутреннее представление в виде массива целых чисел. В процессе выполнения алгоритмов происходит последовательное изменение элементов массива, которое должно корректно отображаться средствами визуализации без влияния на саму логику сортировки.

Анализ требований показал необходимость реализации модульной архитектуры с использованием интерфейсов, обеспечивающих взаимодействие между компонентами программы. Это позволяет разделить систему на независимые части: алгоритмы сортировки, модули загрузки и сохранения данных, а также модули визуализации. Такой подход упрощает сопровождение программы и позволяет расширять её функциональность без изменения существующего кода.

Важным требованием является использование паттернов проектирования. В рамках проекта применяются паттерны Factory для создания объектов (алгоритмов, визуализации и работы с данными), Strategy для выбора алгоритма сортировки, а также Observer для передачи изменений состояния массива в модуль визуализации. Использование данных паттернов обеспечивает гибкость архитектуры и независимость компонентов системы.

					МИВУ.09.03.02 020 ПЗ	Лист
						8
Изм.	Лист	№ докум.	Подпись	Дата		

Также в техническом задании предусмотрена возможность выбора алгоритма сортировки, способа хранения данных и варианта визуализации. Это реализуется за счёт использования фабрик и интерфейсов, позволяющих динамически определять необходимые компоненты во время выполнения программы. Такой подход делает систему универсальной и адаптируемой под различные сценарии использования.

Особое внимание уделено требованию расширяемости. Архитектура программы спроектирована таким образом, чтобы добавление новых алгоритмов сортировки, форматов данных или способов визуализации не требовало изменения уже реализованных модулей. Для этого достаточно создать новый класс, реализующий соответствующий интерфейс, и зарегистрировать его в фабрике.

В результате анализа технического задания определены основные компоненты системы, их взаимодействие и принципы построения архитектуры. Выбранный подход, основанный на использовании интерфейсов и паттернов проектирования, полностью соответствует поставленным требованиям и обеспечивает гибкость, расширяемость и наглядность работы приложения.

					МИВУ.09.03.02 020 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

2. Проектирование программы

После проведения анализа технического задания была определена общая архитектура разрабатываемой системы. Основной задачей проекта являлось создание программы визуализации алгоритмов сортировки, поддерживающей несколько алгоритмов, различные способы отображения данных и работу с несколькими форматами файлов. Дополнительно необходимо было обеспечить независимость алгоритмов сортировки от системы визуализации и возможность дальнейшего расширения программы без изменения существующей логики.

На начальном этапе разработки была сформирована последовательность проектирования основных компонентов системы. В процессе разработки выполнялось создание структуры проекта, проектирование интерфейсов, реализация алгоритмов сортировки, разработка системы визуализации, настройка работы с файлами, подключение системы сборки Maven, а также тестирование и анализ качества кода.

На рисунке 1 представлена общая диаграмма этапов разработки программы визуализации алгоритмов сортировки.

					МИВУ.09.03.02 020 ПЗ	Лист
						10
Изм.	Лист	№ докум.	Подпись	Дата		

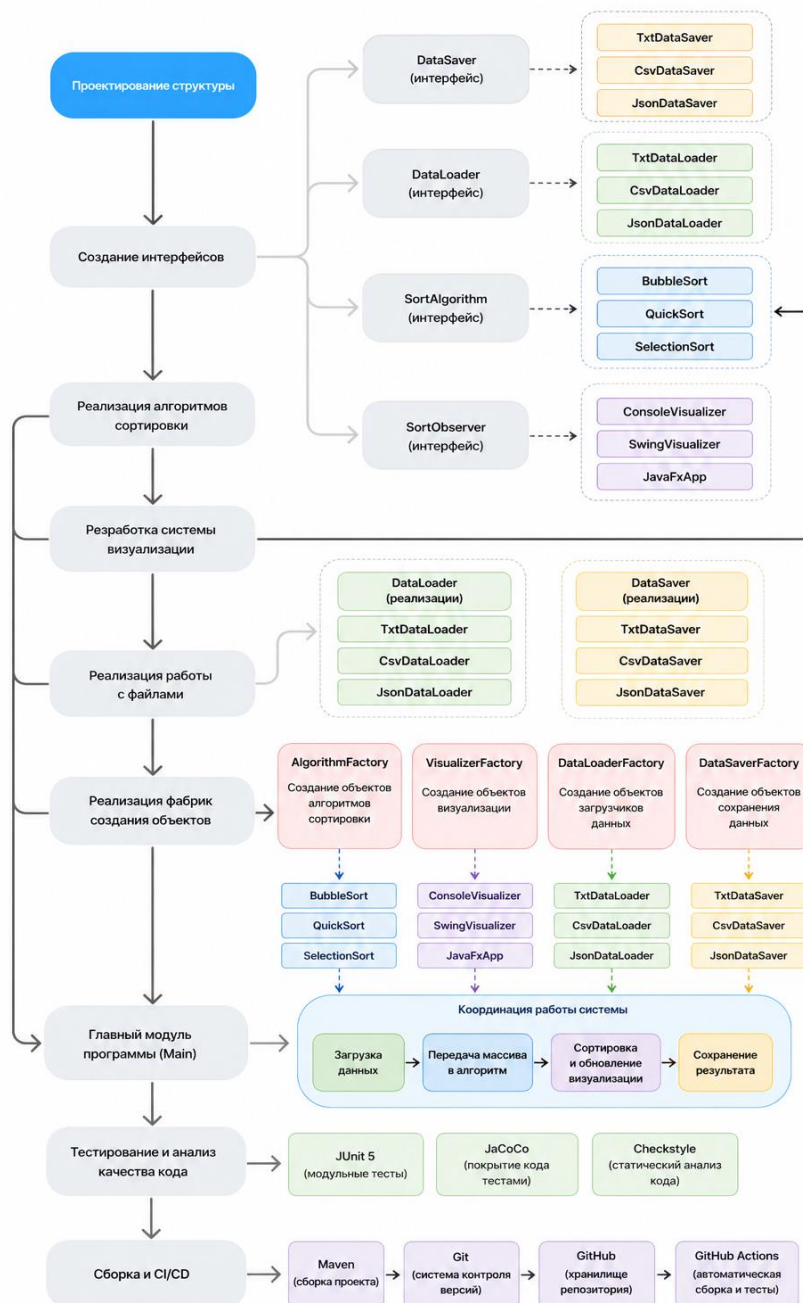


Рисунок 1 – Диаграмма этапов разработки программы

2.1 Проектирование системы на Java

При проектировании системы было принято решение использовать объектно-ориентированный подход и модульную архитектуру. Основной целью являлось разделение программы на независимые подсистемы, каждая из которых

отвечает только за собственную задачу.

В процессе проектирования были выделены следующие основные подсистемы:

- подсистема алгоритмов сортировки;
- подсистема визуализации;
- подсистема хранения данных;
- подсистема управления объектами программы.

Для обеспечения взаимодействия между компонентами были спроектированы интерфейсы, определяющие общий механизм работы различных модулей. Такой подход позволил сделать архитектуру гибкой и обеспечить возможность дальнейшего расширения программы.

Подсистема алгоритмов сортировки проектировалась как независимый модуль, работающий только с массивом данных, находящимся в оперативной памяти. При этом алгоритмы не должны были зависеть от графического интерфейса или формата входных файлов.

Для организации взаимодействия алгоритмов и визуализации был предусмотрен механизм передачи состояния массива в систему отображения данных. Благодаря этому визуализация должна была получать только информацию о текущем состоянии массива, не влияя на сам процесс сортировки.

Также на этапе проектирования была предусмотрена система фабрик объектов. Использование фабрик позволило централизовать создание компонентов программы и исключить зависимость основного модуля от конкретных реализаций алгоритмов, визуализаторов и подсистем хранения данных.

2.2 Проектирование системы хранения данных

Одним из требований технического задания являлась поддержка нескольких форматов файлов. В связи с этим при проектировании была

					МИВУ.09.03.02 020 ПЗ	Лист
						12
Изм.	Лист	№ докум.	Подпись	Дата		

предусмотрена отдельная подсистема хранения данных.

Основной задачей данной подсистемы являлось обеспечение универсального механизма загрузки и сохранения информации независимо от формата файла. Для этого было принято решение использовать отдельные интерфейсы загрузки и сохранения данных.

На этапе проектирования были определены поддерживаемые форматы файлов:

- TXT;
- CSV;
- JSON.

Внутри программы данные должны были храниться в виде массива целых чисел. Поэтому независимо от формата входного файла информация после загрузки преобразуется во внутреннее представление в оперативной памяти.

При проектировании системы хранения также учитывалась возможность дальнейшего расширения. Архитектура должна позволять добавление новых форматов файлов без изменения существующей логики программы. Для этого каждый формат данных проектировался как отдельный независимый модуль, взаимодействующий с системой через общий интерфейс.

2.3 Проектирование графического интерфейса

При проектировании графического интерфейса основной задачей являлось создание наглядной системы визуализации алгоритмов сортировки и обеспечение удобного взаимодействия пользователя с программой.

На этапе проектирования было принято решение реализовать несколько вариантов визуализации:

- консольную визуализацию;
- графический интерфейс на основе Swing;
- графический интерфейс на основе JavaFX.

Использование нескольких вариантов визуализации позволило обеспечить универсальность программы и сравнить различные подходы к отображению данных.

Консольная визуализация проектировалась как наиболее простой способ отображения работы алгоритмов. Основной задачей данного варианта являлся вывод текущего состояния массива в текстовом виде после выполнения операций сортировки. Дополнительно предусматривалась подсветка изменяемых элементов массива для повышения наглядности отображения процесса сортировки в терминале.

Графический интерфейс Swing проектировался как облегчённая оконная система визуализации. В рамках данного интерфейса предусматривалось отображение массива в виде столбчатой диаграммы, обновляемой во время выполнения сортировки. Swing-визуализация должна была обеспечить более наглядное представление работы алгоритмов по сравнению с консольным вариантом, сохранив при этом относительно простую структуру реализации.

Основной графической подсистемой проекта была выбрана JavaFX, поскольку данная технология предоставляет более современные средства разработки пользовательских интерфейсов и анимации. При проектировании интерфейса JavaFX были определены основные элементы управления:

- выбор входного файла;
- выбор выходного файла;
- выбор алгоритма сортировки;
- настройка скорости визуализации;
- область отображения массива.

Визуализация массива проектировалась в виде столбчатой диаграммы, где высота каждого элемента соответствует значению числа в массиве. Дополнительно была предусмотрена подсветка элементов массива во время выполнения сортировки для повышения наглядности отображения работы алгоритмов.

Особое внимание при проектировании интерфейса уделялось

независимости визуализации от логики сортировки. Графический интерфейс должен был только отображать состояние массива, не вмешиваясь в выполнение алгоритма сортировки. Благодаря этому обеспечивается единый механизм работы программы независимо от используемой библиотеки визуализации.

					МИВУ.09.03.02 020 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

3. Разработка приложения

3.1 Разработка системы на Java

Основой архитектуры программы стало использование объектно-ориентированного подхода и интерфейсов. Применение интерфейсов позволило разделить систему на независимые компоненты и обеспечить взаимодействие между ними через общий набор методов.

При проектировании было принято решение разделить программу на несколько основных подсистем:

- подсистема алгоритмов сортировки;
- подсистема визуализации;
- подсистема работы с файлами;
- подсистема хранения данных;
- подсистема хранения объектов.

Такое разделение позволило уменьшить связанность компонентов и упростить дальнейшее расширение проекта.

Для реализации алгоритмов сортировки был создан интерфейс `SortAlgorithm`, определяющий общий метод сортировки массива данных. Использование интерфейса позволило организовать единый механизм взаимодействия между программой и различными алгоритмами сортировки. интерфейс `SortAlgorithm` представлен на рисунке 2.

```
public interface SortAlgorithm {  
    void sort(int[] array, SortObserver observer);  
}
```

Рисунок 2 – Интерфейс `SortAlgorithm`

Интерфейс реализует классы `BubbleSort`, `QuickSort`, `SelectionSort`. Данные классы представлены на рисунках 3-5.

```

public class BubbleSort implements SortAlgorithm { 1 usage  👤 admin +1 *
    @Override 4 usages  👤 admin +1 *
    public void sort(int[] array, SortObserver observer) {
        int n = array.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                observer.update(array.clone(), j, index2: j + 1);
                if (array[j] > array[j + 1]) {
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    observer.update(array.clone(), j, index2: j + 1);
                }
            }
        }
    }
}

```

Рисунок 3 – Класс BubbleSort

```

public class QuickSort implements SortAlgorithm { 1 usage  👤 admin +1 *
    @Override 4 usages  👤 admin
    public void sort(int[] array, SortObserver observer) { quickSort(array, low: 0, high: array.length - 1, observer);
    private void quickSort(int[] array, int low, int high, SortObserver observer) { 3 usages  👤 admin

        if (low < high) {
            int pi = partition(array, low, high, observer);
            quickSort(array, low, high: pi - 1, observer);
            quickSort(array, low: pi + 1, high, observer);
        }
    }
}

```

Рисунок 4 – Класс QuickSort

```

public class SelectionSort implements SortAlgorithm { 2 usages  👤 admin +1 *
    @Override 4 usages  👤 admin +1 *
    public void sort(int[] array, SortObserver observer) {
        int n = array.length;
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;
            for (int j = i + 1; j < n; j++) {
                observer.update(array.clone(), j, minIndex);
                if (array[j] < array[minIndex]) {
                    minIndex = j;
                    observer.update(array.clone(), j, minIndex);
                }
            }

            if (minIndex != i) {
                int temp = array[i];
                array[i] = array[minIndex];
                array[minIndex] = temp;

                observer.update(array.clone(), i, minIndex);
            }
        }
    }
}

```

Рисунок 5 – Класс SelectionSort

Каждый из данных классов содержит собственную реализацию алгоритма сортировки, однако все они работают через единый интерфейс. Благодаря этому основной код программы не зависит от конкретного алгоритма и может динамически выбирать нужную реализацию.

Особое внимание при проектировании уделялось независимости алгоритмов сортировки от системы отображения данных. Для решения данной задачи был реализован интерфейс SortObserver. Через него алгоритмы передают информацию о текущем состоянии массива в модуль визуализации.

Интерфейс SortObserver реализует классы ConsoleVisualizer, SwingVisualizer, JavaFxApp. Данные классы представлены на рисунках 6-8.

```
public class ConsoleVisualizer implements SortObserver { 2 usages  darya +

    @Override 9 usages  darya +1 *
    public void update(int[] array, int i, int j) {
        clearConsole();
        for (int k = 0; k < array.length; k++) {...}
        System.out.println();
        drawBars(array, i, j);
        try {
            Thread.sleep(100);
        } catch (InterruptedException ignored) {}
    }

    private void drawBars(int[] array, int i, int j) { 1 usage  darya *
        int max = java.util.Arrays.stream(array).max().orElse(1);
        for (int level = max; level > 0; level--) {...}
        System.out.println();
    }

    private void clearConsole() { 1 usage  darya
        System.out.print("\033[H\033[2J");
        System.out.flush();
    }
}
```

Рисунок 6 – Класс ConsoleVisualizer

```

public class SwingVisualizer extends JFrame implements SortObserver {

    private int[] data; 11 usages
    private String outputPath; 4 usages
    private final DrawPanel drawPanel = new DrawPanel(); 3 usages
    private JComboBox<String> algorithmBox; 3 usages
    private JSlider speedSlider; 6 usages
    private int index1 = -1; 2 usages
    private int index2 = -1; 2 usages

    public SwingVisualizer() { 1 usage  @darya

        setTitle("Sorting Visualizer (Swing)");
        setSize( width: 900, height: 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        initUI();

        setVisible(true);
    }

    private void initUI() {...}

    @Override 9 usages  @darya
    public void update(int[] array, int i, int j) {...}

    class DrawPanel extends JPanel {...}
}

```

Рисунок 7 – Класс SwingVisualizer

```

public class JavaFxApp extends Application implements SortObserver {

    private int[] data; 5 usages
    private String outputPath; 6 usages

    private Pane drawPane = new Pane(); 9 usages
    private List<Rectangle> bars = new ArrayList<>(); 2 usages

    private Slider speedSlider = new Slider(v: 1, v1: 100, v2: 50); 4

    @Override  @darya
    public void start(Stage stage) {...}

    private void drawArray(int[] array, int index1, int index2) { 2 usages
        Platform.runLater(() -> {...});
    }

    @Override 9 usages  @darya
    public void update(int[] array, int i, int j) {...}

    private void showAlert(String text) {...}
}

```

Рисунок 8 – Класс JavaFxApp

Во время работы алгоритм сортировки изменяет массив в оперативной памяти и после выполнения значимых операций вызывает метод обновления визуализации. В этот момент визуализатор получает текущее состояние массива и отображает изменения пользователю. Таким образом, сортировка не зависит от конкретной библиотеки графического интерфейса и работает одинаково для всех вариантов визуализации.

Для централизованного создания объектов были реализованы фабрики AlgorithmFactory, VisualizerFactory, DataLoaderFactory, DataSaverFactory. Данные фабрики представлены на рисунках 9-12.

```
public class AlgorithmFactory { 3 usages  admin

    public static SortAlgorithm create(String name) { 3 usages  admin

        switch (name.toLowerCase()) {

            case "bubble":
                return new BubbleSort();
            case "selection":
                return new SelectionSort();
            case "quick":
                return new QuickSort();
            default:
                throw new IllegalArgumentException("Неизвестный алгоритм: " + name);
        }
    }
}
```

Рисунок 9 – Фабрика AlgorithmFactory

```
import org.example.algorithm.SortObserver;
import org.example.visualization.ConsoleVisualizer;
import org.example.visualization.SwingVisualizer;

public class VisualizerFactory { 1 usage  admin +1

    public static SortObserver create(String type, int[] data) { 1 usage  admin +1

        switch (type.toLowerCase()) {

            case "console":
                return new ConsoleVisualizer();
            case "swing":
                return new SwingVisualizer();
            default:
                throw new IllegalArgumentException("Неизвестный тип визуализации: " + type);
        }
    }
}
```

Рисунок 10 – Фабрика VisualizerFactory

```

public class DataLoaderFactory { 3 usages  admin *

    public static DataLoader create(String filePath) { 3 usages  admin *
        if (filePath.endsWith(".json")) {
            return new JsonDataLoader();
        }
        if (filePath.endsWith(".csv")) {
            return new CsvDataLoader();
        }
        if (filePath.endsWith(".txt")) {
            return new TxtDataLoader();
        }
        throw new IllegalArgumentException("Неподдерживаемый формат файла: " + filePath);
    }
}

```

Рисунок 11 – Фабрика DataLoaderFactory

```

public class DataSaverFactory { 3 usages  darya *

    public static DataSaver create(String filePath) { 3 usages  darya *
        if (filePath.endsWith(".json")) {
            return new JsonDataSaver();
        }
        if (filePath.endsWith(".csv")) {
            return new CsvDataSaver();
        }
        if (filePath.endsWith(".txt")) {
            return new TxtDataSaver();
        }

        throw new IllegalArgumentException("Неподдерживаемый формат файла: " + filePath);
    }
}

```

Рисунок 12 – Фабрика DataSaverFactory

Использование фабрик позволило убрать из основного кода программы большое количество условных операторов и реализовать централизованное создание объектов. Основной модуль программы взаимодействует не с конкретными реализациями классов, а с интерфейсами, что значительно упрощает расширение системы.

Главным координирующим компонентом программы является класс Main, представленный на рисунке 13.


```

public class Main {
    public static void main(String[] args) {
        if (args.length == 1 && args[0].equals("--help")) {...}
        if (args.length != 4) {...}
        String inputFile = args[0];
        String algorithmName = args[1];
        String visualizationType = args[2];
        String outputFile = args[3];

        // Загрузка данных
        DataLoader loader = DataLoaderFactory.create(inputFile);
        int[] data = loader.load(inputFile);

        // Выбор алгоритма
        SortAlgorithm algorithm = AlgorithmFactory.create(algorithmName);

        // Выбор визуализации
        SortObserver visualizer = VisualizerFactory.create(visualizationType, data);

        // Сортировка
        algorithm.sort(data, visualizer);

        // Сохранение результата
        DataSaver saver = DataSaverFactory.create(outputFile);
        saver.save(data, outputFile);

        System.out.println("Сортировка завершена.");
    } catch (IllegalArgumentException e) {...} catch (Exception e) {
        System.out.println("Произошла непредвиденная ошибка: " + e.getMessage());
    }
}

```

Рисунок 12 – Класс Main

Класс Main отвечает за получение параметров запуска программы, создание необходимых объектов через фабрики и организацию взаимодействия между всеми компонентами системы. Именно данный класс связывает между собой загрузку данных, выбор алгоритма сортировки, визуализацию процесса сортировки и сохранение результата.

3.2 Разработка системы хранения данных

Следующим этапом стало проектирование подсистемы хранения и обработки данных. Поскольку программа должна поддерживать несколько форматов входных и выходных файлов, было принято решение реализовать отдельную систему загрузки и сохранения данных. Для этого были созданы интерфейсы DataLoader, DataSaver, DataStorage. Данные интерфейсы представлены на рисунках 13-15.

```
public interface DataLoader {
    int[] load(String filePath);
}
```

Рисунок 13 – Интерфейс DataLoader

```
public interface DataSaver {
    void save(int[] array, String filePath);
}
```

Рисунок 14 – Интерфейс DataSaver

```
public interface DataStorage {
    int[] loadData(String path);
    void saveData(String path, int[] data);
}
```

Рисунок 15 – Интерфейс DataStorage

Следующим этапом стало проектирование подсистемы хранения и обработки данных. Поскольку программа должна поддерживать несколько форматов входных и выходных файлов, было принято решение реализовать отдельную систему загрузки и сохранения данных. Для этого были созданы интерфейсы `DataLoader`, `DataSaver`, `DataStorage`. Данные интерфейсы представлены на рисунках 13-15.

Интерфейс `DataLoader` отвечает за загрузку массива данных из файла, интерфейс `DataSaver` — за сохранение результата сортировки, а `DataStorage` объединяет механизмы загрузки и сохранения данных.

Для поддержки различных форматов файлов были реализованы следующие классы:

- `TxtDataLoader`;
- `CsvDataLoader`;
- `JsonDataLoader`;
- `TxtDataSaver`;
- `CsvDataSaver`;
- `JsonDataSaver`.

- TxtStorage;
- CsvStorage;
- JsonStorage.

Работа подсистемы хранения данных организована следующим образом: пользователь выбирает входной файл, после чего через фабрику создаётся соответствующий загрузчик данных. Загрузчик считывает информацию из файла и преобразует её в массив `int[]`, находящийся в оперативной памяти. После завершения сортировки массив передаётся классу сохранения данных и записывается в файл необходимого формата.

Использование интерфейсов позволило полностью отделить работу с файлами от логики сортировки и визуализации. Благодаря этому добавление нового формата хранения данных требует только создания нового класса, реализующего соответствующий интерфейс.

3.3 Разработка графического интерфейса

После завершения проектирования основной логики программы была разработана система визуализации и графический интерфейс приложения. Основной задачей данного этапа являлось создание наглядного отображения процесса сортировки и обеспечение удобного взаимодействия пользователя с программой.

В проекте реализованы три варианта визуализации:

- консольная визуализация;
- визуализация на основе Swing;
- визуализация на основе JavaFX.

Консольная визуализация реализована в виде текстового отображения массива в терминале. Во время работы алгоритма после каждого значимого изменения состояния массива выполняется вывод его текущего состояния. Для повышения наглядности отображения была реализована подсветка изменяемых

элементов массива средствами ANSI-цветов терминала. Данный вариант визуализации отличается минимальными требованиями к ресурсам системы и используется для тестирования алгоритмов сортировки, а также для запуска программы без графического интерфейса.

Графический интерфейс Swing реализован в виде отдельного окна визуализации, отображающего массив в форме столбчатой диаграммы. Высота каждого столбца соответствует значению элемента массива. Во время сортировки элементы массива обновляются в режиме реального времени, а сравниваемые элементы подсвечиваются отдельным цветом.

Наиболее функциональной частью проекта является визуализация на основе JavaFX. Интерфейс JavaFX включает:

- выбор входного файла;
- выбор выходного файла;
- выбор алгоритма сортировки;
- настройку скорости анимации;
- графическое отображение массива;
- подсветку элементов;
- отображение значений массива.

На рисунках 16-18 представлено главное окно JavaFX-приложения. Интерфейс включает элементы управления загрузкой входного файла, выбором алгоритма сортировки, настройкой скорости визуализации и выбором выходного файла. Центральная часть окна используется для графического отображения массива в процессе сортировки.

					МИВУ.09.03.02 020 ПЗ	Лист
						25
Изм.	Лист	№ докум.	Подпись	Дата		

```

@Override @darya
public void start(Stage stage) {

    Button fileButton = new Button(s: "Выбрать входной файл");
    Button saveButton = new Button(s: "Выбрать выходной файл");

    TextField outputField = new TextField();
    outputField.setPromptText("Файл для сохранения результата");

    ComboBox<String> algorithmBox = new ComboBox<>();
    Button startButton = new Button(s: "Начать сортировку");

```

Рисунок 16 – Главное окно

```

Label speedLabel = new Label(s: "Скорость");
drawPane.setPrefSize(v: 800, v1: 500);
drawPane.setStyle(
    "-fx-background-color:#f4f4f4;" +
    "-fx-border-color:#cccccc;" +
    "-fx-border-width:1;"
);

algorithmBox.getItems().addAll(es: "quick", "bubble", "selection");
algorithmBox.setValue("quick");

```

Рисунок 17 – Главное окно

```

fileButton.setOnAction(ActionEvent e -> {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Выбор файла");

    File file = fileChooser.showOpenDialog(stage);

    if (file != null) {
        DataLoader loader = DataLoaderFactory.create(file.getName());
        data = loader.load(file.getAbsolutePath());
        drawArray(data, index1: -1, index2: -1);
    }
});

saveButton.setOnAction(ActionEvent e -> {
    FileChooser fileChooser = new FileChooser();
    fileChooser.setTitle("Сохранить файл");

    File file = fileChooser.showSaveDialog(stage);

    if (file != null) {
        outputPath = file.getAbsolutePath();
        outputField.setText(outputPath);
    }
});

```

Рисунок 18 – Главное окно

На рисунке 19 представлена визуализация массива в процессе сортировки. Элементы массива отображаются в виде столбчатой диаграммы, где высота каждого столбца соответствует значению элемента. Сравнимые элементы массива подсвечиваются отдельным цветом, а над столбцами отображаются числовые значения элементов. Обновление визуализации выполняется после каждого значимого шага алгоритма сортировки через интерфейс SortObserver.

```
private void drawArray(int[] array, int index1, int index2) { 2 usages darya *

Platform.runLater() -> {

    drawPane.getChildren().clear();
    bars.clear();
    double paneWidth = drawPane.getWidth() > 0 ? drawPane.getWidth() : 800;
    double paneHeight = drawPane.getHeight() > 0 ? drawPane.getHeight() : 500;
    double topPadding = 40; // пространство сверху
    paneHeight -= topPadding;
    double width = paneWidth / array.length;
    int max = java.util.Arrays.stream(array).max().orElse(1);
    for (int i = 0; i < array.length; i++) {
        double height = (array[i] * paneHeight) / max;
        Rectangle rect = new Rectangle(
            v: i * width,
            v1: paneHeight - height + topPadding,
            v2: width - 4,
            height
        );
        if (i == index1 || i == index2) {
            rect.setFill(Color.web(s: "#ff6b6b")); // мягкий красный
        } else {
            rect.setFill(Color.web(s: "#5c7cfa")); // мягкий синий
        }
        bars.add(rect);
        Text text = new Text(
            v: i * width + width / 4,
            v1: paneHeight - height + topPadding - 5,
            String.valueOf(array[i])
        );
        drawPane.getChildren().addAll(rect, text);
    }
}
```

Рисунок 19 – Визуализация массива в процессе сортировки

Для реализации графического интерфейса использовались компоненты JavaFX:

- Stage — главное окно приложения JavaFX;
- Scene — содержимое окна, в котором размещаются все элементы интерфейса;
- Pane — область для размещения и отрисовки графических элементов

визуализации.;

- VBox — контейнер для вертикального расположения элементов интерфейса;
- HBox — контейнер для горизонтального расположения элементов интерфейса;
- Button — кнопка для выполнения действий пользователя;
- Slider — элемент управления для настройки скорости визуализации сортировки;
- ComboBox — выпадающий список для выбора алгоритма сортировки;
- Label — текстовая подпись для отображения названий и пояснений элементов интерфейса.

Отображение массива реализовано с использованием графических объектов Rectangle, формирующих столбчатую диаграмму. Во время сортировки визуализатор получает текущее состояние массива через интерфейс SortObserver и выполняет обновление графических элементов.

При проектировании графического интерфейса особое внимание уделялось независимости визуализации от логики сортировки. Визуализаторы только отображают текущее состояние массива и не влияют на выполнение алгоритмов сортировки. Благодаря этому программа обеспечивает одинаковую работу алгоритмов независимо от используемой системы визуализации.

В результате проектирования была разработана модульная архитектура программы с разделением ответственности между компонентами системы. Использование интерфейсов и паттернов проектирования позволило обеспечить гибкость архитектуры, независимость модулей и возможность дальнейшего расширения функциональности приложения.

4. Тестирование

Одним из важнейших этапов создания программного продукта является его тестирование и отладка. Тестирование позволяет выявить скрытые и явные недостатки программы, либо убедиться в ее пригодности для применения.

Целью тестирования является проверка работоспособности программы, то есть правильности выполнения всех функций, описанных выше.

Таблица 1 – Тестирование

№	Действие	Результат
1	2	3
1.	Запуск функции help	Успех, корректное отображение информации
2.	Попытка передать несуществующий параметр алгоритма сортировки	Вывод сообщения «Неизвестный алгоритм: wrong»
3.	Попытка запуска программы с консольной визуализацией	Успех, сортировка массива и сохранение результата в указанный файл
4.	Попытка запуска программы с визуализацией через библиотеку Swing	Успех, сортировка массива
5.	Попытка сохранения результата с визуализацией через библиотеку Swing	Успех, сохранение файла
6.	Попытка запуска программы с визуализацией через библиотеку JavaFX	Успех
7.	Попытка запуска программы с выбором алгоритма сортировки и файлами для загрузки и сохранения результата	Успех, сортировка массива и сохранение файла
8.	Попытка провести maven-тесты	Успех, выполнение всех тестов
9.	Попытка провести статический анализ	Успех

Вывод сообщений об ошибках и другие сообщения представлены на рисунках 14-24.

```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar --help
Использование:
java -jar app.jar <input> <algorithm> <visualization> <output>

Пример:
java -jar app.jar forjson.json quick console result.json

Поддерживаемые форматы входа: .json .csv .txt
Поддерживаемые форматы выхода: .json .csv .txt
Алгоритмы: quick, bubble, selection
```

Рисунок 14 – Вывод результата теста №1

```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar forjson.json wrong console result.json
Ошибка: Неизвестный алгоритм: wrong
```

Рисунок 15 – Вывод результата теста №2



Рисунок 16 – Вывод результата теста №3

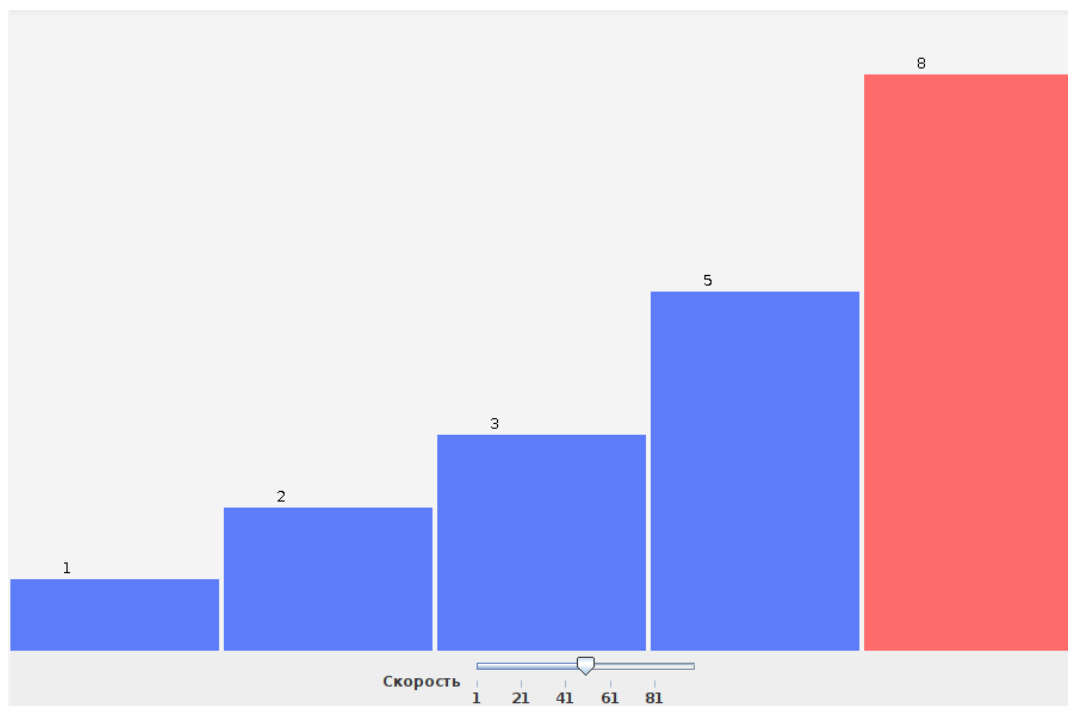


Рисунок 17 – Вывод результата теста №4

```
[admin@localhost smakotina_kyrsovaya_java]$ java -jar target/smakotina_kyrsovaya_java-1.0-SNAPSHOT.jar forjson.json quick swing output.json
Результат сохранён в файл: output.json
Сортировка завершена.
█
```

Рисунок 18 – Вывод результата теста №5

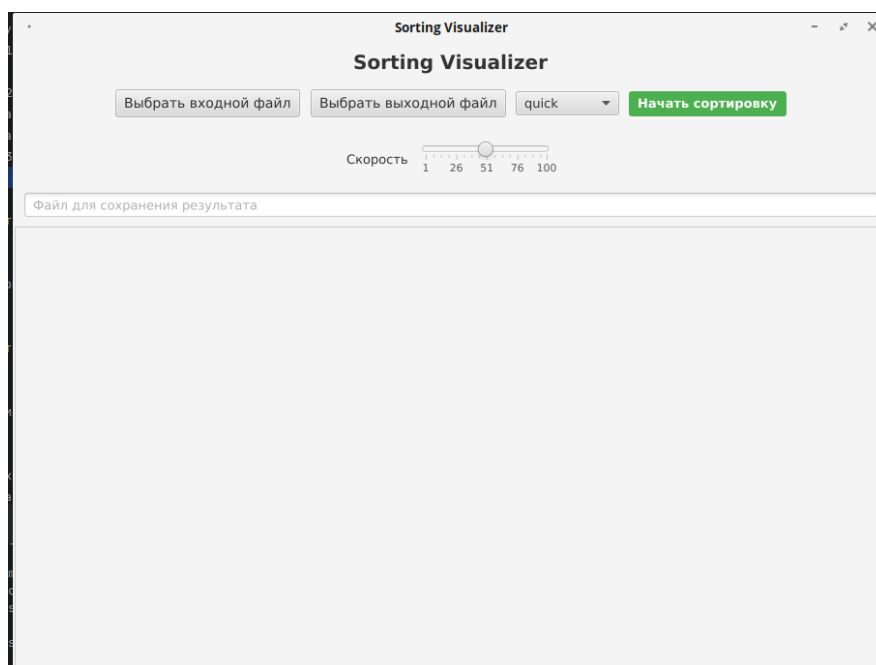


Рисунок 19 – Вывод результата теста №6

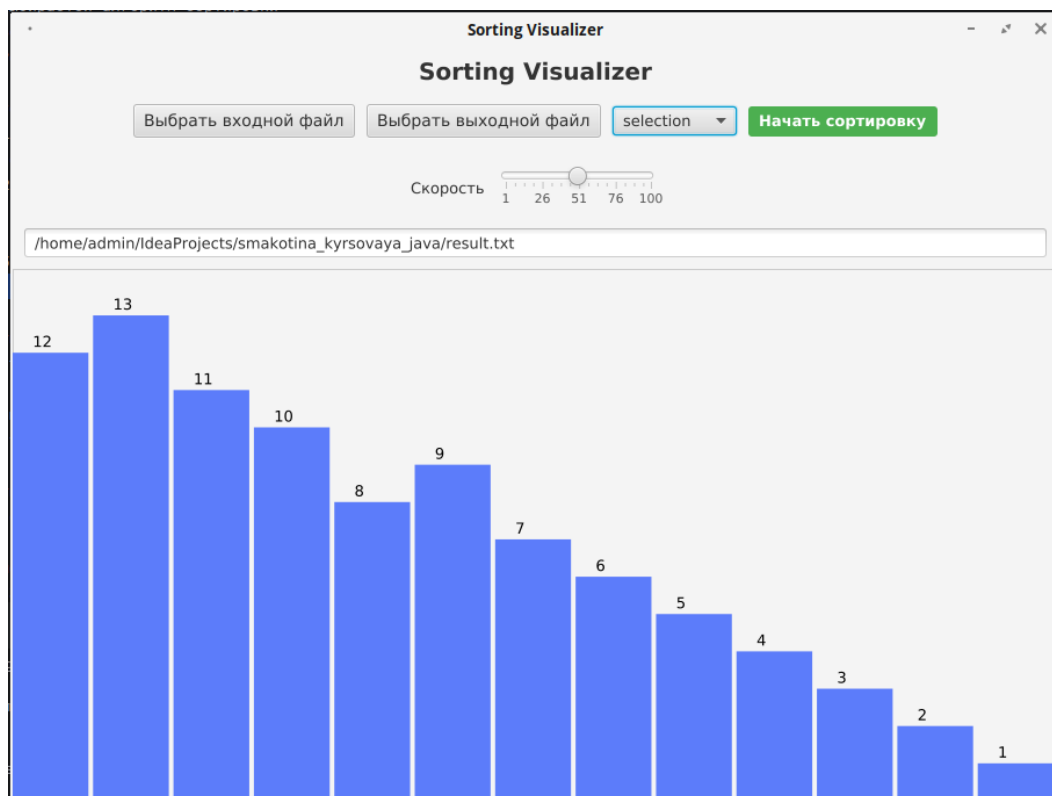


Рисунок 20 – Вывод результата теста №7

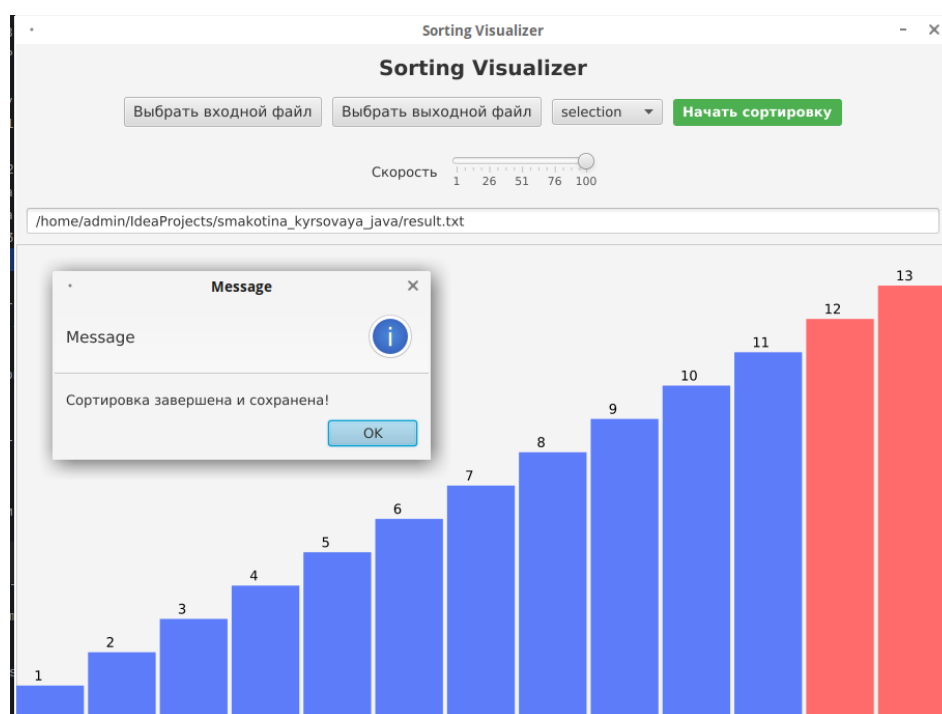


Рисунок 21 – Вывод результата теста №7

Результат сохранён в файл: /home/admin/IdeaProjects/smakotina_kyrsovaya_java/result.txt [1/50]

Рисунок 22 – Вывод результата теста №7

```

[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running org.example.algorithm.SortTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.097 s -- in org.example.algorithm.SortTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ smakotina_kyrsovaya_java ---
[INFO] Loading execution data file /home/admin/IdeaProjects/smakotina_kyrsovaya_java/target/jacoco.exec
[INFO] Analyzed bundle 'smakotina_kyrsovaya_java' with 26 classes
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO]
[INFO] Total time: 3.393 s
[INFO] Finished at: 2026-05-13T10:02:00+03:00
[INFO] -----

```

Рисунок 23 – Вывод результата теста №8

```

[admin@localhost smakotina_kyrsovaya_java]$ mvn checkstyle:check
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.example:smakotina_kyrsovaya_java >-----
[INFO] Building smakotina_kyrsovaya_java 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- checkstyle:3.3.1:check (default-cli) @ smakotina_kyrsovaya_java ---
[INFO] You have 0 Checkstyle violations.
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.227 s
[INFO] Finished at: 2026-05-13T10:05:56+03:00
[INFO]

```

Рисунок 24 – Вывод результата теста №9

Заключение

В ходе выполнения курсовой работы была разработана программная система визуализации алгоритмов сортировки на языке Java. Основной целью проекта являлось создание расширяемого приложения, способного выполнять сортировку данных, загруженных из файлов различных форматов, и отображать процесс сортировки несколькими способами визуализации.

В процессе разработки была спроектирована модульная архитектура программы, включающая подсистемы алгоритмов сортировки, визуализации, загрузки и сохранения данных, а также систему фабрик для создания объектов. Такой подход позволил разделить ответственность между компонентами программы и обеспечить их независимость друг от друга.

В рамках работы были реализованы алгоритмы пузырьковой сортировки, быстрой сортировки и сортировки выбором. Для взаимодействия компонентов использовались интерфейсы SortAlgorithm, SortObserver, DataLoader и DataSaver. Сортировка выполняется в оперативной памяти, а визуализация только отображает текущее состояние массива, не влияя на работу алгоритмов.

Для отображения процесса сортировки были реализованы три варианта визуализации: консольная визуализация, графическая визуализация на основе Swing и JavaFX. Также была добавлена поддержка форматов TXT, CSV и JSON для загрузки и сохранения данных.

При разработке были использованы паттерны проектирования Factory, Observer и Strategy, что обеспечило гибкость архитектуры и возможность дальнейшего расширения программы. Дополнительно были настроены Maven, GitHub Actions, модульное тестирование с использованием JUnit.

В результате выполнения курсовой работы была создана полнофункциональная система визуализации алгоритмов сортировки, соответствующая поставленным требованиям и демонстрирующая применение

принципов объектно-ориентированного программирования и современных средств разработки программного обеспечения.

					МИВУ.09.03.02 020 ПЗ	Лист
						35
Изм.	Лист	№ докум.	Подпись	Дата		

Библиографи

1. Ермаков, А. В. Разработка приложений на языке JAVA : учебное пособие / А. В. Ермаков, М. С. Королёв, А. К. Кузьмин. — Саратов : Саратовский государственный технический университет имени Ю.А. Гагарина, ЭБС АСВ, 2024. — 128 с. — ISBN 978-5-7433-3635-7. — Текст : электронный // Цифровой образовательный ресурс IPR SMART : [сайт]. — URL: <https://www.iprbookshop.ru/150088.html>
2. Java. Библиотека профессионала. Том 1. Основы [Электронный ресурс]: / К. С. Хорстманн, Г. Корнелл. — Электрон. текстовые данные. — М.: Вильямс, 2019. — 832 с. — Режим доступа: <https://www.litres.ru/key-horstmann/java-biblioteka-professionala-tom-1-osnovy/>
3. Oracle. JavaFX Documentation [Электронный ресурс]. — Режим доступа: https://openjfx.io/?utm_source=chatgpt.com
4. Oracle. Java Swing Tutorial [Электронный ресурс]. — Режим доступа: https://docs.oracle.com/javase/tutorial/uiswing/?utm_source=chatgpt.com

					МИВУ.09.03.02 020 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		36

